

Exercise Sheet 9

Name: Aryan Varshneya

Matriculation Number: 261678

7.4: Measuring Calibration

1. Compute the ECE of each of your three trained CARLA models on the in-distribution test set. 2. Plot a reliability diagram (confidence vs. accuracy) for each model. 3. Are your models over- or underconfident? Does this pattern hold consistently across all three models?

```
In [1]: # Cell 1
import torch
import torch.nn as nn

from torchvision import models, transforms
from torch.utils.data import Dataset, DataLoader

from PIL import Image

import pandas as pd
import numpy as np

import matplotlib.pyplot as plt

from sklearn.calibration import calibration_curve
```

```
In [2]: # Cell 2
device = torch.device(
    "cuda"
    if torch.cuda.is_available()
    else "cpu"
)

print(device)
```

cuda

```
In [3]: # Cell 3
class CarlaDataset(Dataset):

    def __init__(
        self,
        csv_file,
        img_dir,
        label_column=None,
        transform=None
    ):

        self.labels = pd.read_csv(csv_file)
```

```

        self.img_dir = img_dir

        self.label_column = label_column

        self.transform = transform

    def __len__(self):

        return len(self.labels)

    def __getitem__(self, idx):

        row = self.labels.iloc[idx]

        frame = str(row["frame"]).zfill(6)

        img_path = f"{self.img_dir}/{frame}.jpg"

        image = Image.open(
            img_path
        ).convert("RGB")

        if self.transform:

            image = self.transform(image)

        if self.label_column:

            label = float(
                row[self.label_column]
            )

            return image, torch.tensor(label)

        return image

```

```

In [4]: # Cell 4
transform = transforms.Compose([

    transforms.Resize((224,224)),

    transforms.ToTensor(),

    transforms.Normalize(
        mean=[0.485,0.456,0.406],
        std=[0.229,0.224,0.225]
    )
])

```

```

In [5]: # Cell 5
def load_model(model_path):
    model = models.resnet18(
        weights=None
    )
    model.fc = nn.Linear(
        model.fc.in_features,
        1
    )

```

```

    )
    model.load_state_dict(
        torch.load(
            model_path,
            map_location=device
        )
    )
    model = model.to(device)
    model.eval()
    return model

pedestrian_model = load_model(
    "../pedestrian_model.pth"
)
traffic_light_model = load_model(
    "../traffic_light_model.pth"
)
vehicle_model = load_model(
    "../vehicle_model.pth"
)

```

In [6]: # Cell 6

```

pedestrian_dataset = CarlaDataset(
    csv_file="../dataset/test/labels.csv",
    img_dir="../dataset/test/rgb-front",
    label_column="has_pedestrian",
    transform=transform
)

traffic_dataset = CarlaDataset(
    csv_file="../dataset/test/labels.csv",
    img_dir="../dataset/test/rgb-front",
    label_column="has_traffic_light",
    transform=transform
)

vehicle_dataset = CarlaDataset(
    csv_file="../dataset/test/labels.csv",
    img_dir="../dataset/test/rgb-front",
    label_column="has_vehicle",
    transform=transform
)

```

In [7]: # Cell 7

```

pedestrian_loader = DataLoader(
    pedestrian_dataset,
    batch_size=32,
    shuffle=False
)

traffic_loader = DataLoader(
    traffic_dataset,
    batch_size=32,
    shuffle=False
)

vehicle_loader = DataLoader(
    vehicle_dataset,
    batch_size=32,

```

```
    shuffle=False
)
```

```
In [8]: # Cell 8
models_dict = {
    "Pedestrian": (
        pedestrian_model,
        pedestrian_loader
    ),
    "Traffic Light": (
        traffic_light_model,
        traffic_loader
    ),
    "Vehicle": (
        vehicle_model,
        vehicle_loader
    )
}
```

```
In [9]: # Cell 9
def collect_predictions(
    model,
    loader,
    temperature=1.0
):
    model.eval()

    all_probs = []
    all_preds = []
    all_labels = []

    with torch.no_grad():
        for images, labels in loader:

            images = images.to(device)

            logits = model(images)

            logits = logits / temperature

            probs = torch.sigmoid(logits).squeeze()

            preds = (probs >= 0.5).float()

            all_probs.extend(
                probs.cpu().numpy()
            )

            all_preds.extend(
                preds.cpu().numpy()
            )

            all_labels.extend(
                labels.numpy()
            )

    return (
```

```

    np.array(all_probs),
    np.array(all_preds),
    np.array(all_labels)
)

```

```

In [10]: # Cell 10
def compute_ece(
    probs,
    preds,
    labels,
    n_bins=20
):
    # confidence = how sure the model is in its PREDICTED class,
    # not just the raw positive-class probability
    confidences = np.maximum(probs, 1 - probs)

    bins = np.linspace(
        0,
        1,
        n_bins + 1
    )

    ece = 0

    for i in range(n_bins):

        mask = (
            (confidences >= bins[i]) &
            (confidences < bins[i+1])
        )

        if np.sum(mask) == 0:
            continue

        accuracy = np.mean(
            preds[mask] ==
            labels[mask]
        )

        confidence = np.mean(
            confidences[mask]
        )

        weight = (
            np.sum(mask) /
            len(probs)
        )

        ece += (
            weight *
            np.abs(
                accuracy -
                confidence
            )
        )

    return ece

```

```

In [11]: # Cell 11
results = []

for model_name, (
    model,
    loader
) in models_dict.items():

    probs, preds, labels = collect_predictions(
        model,
        loader
    )

    ece = compute_ece(
        probs,
        preds,
        labels
    )

    results.append([
        model_name,
        ece
    ])

ece_df = pd.DataFrame(
    results,
    columns=[
        "Model",
        "ECE"
    ]
)

ece_df

```

```

Out[11]:

```

	Model	ECE
0	Pedestrian	0.107468
1	Traffic Light	0.036597
2	Vehicle	0.046837

```

In [12]: # Cell 12
def plot_reliability_diagram(
    probs,
    labels,
    model_name
):

    true_prob, pred_prob = calibration_curve(
        labels,
        probs,
        n_bins=20,
        strategy="uniform"
    )

    plt.figure(figsize=(6,6))

    plt.plot(

```

```

        pred_prob,
        true_prob,
        marker="o",
        label="Model"
    )

    plt.plot(
        [0,1],
        [0,1],
        "--",
        label="Perfect Calibration"
    )

    plt.xlabel("Mean Predicted Confidence")

    plt.ylabel("Observed Accuracy")

    plt.title(
        f"Reliability Diagram - {model_name}"
    )

    plt.legend()

    plt.grid(True)

    plt.show()

```

```

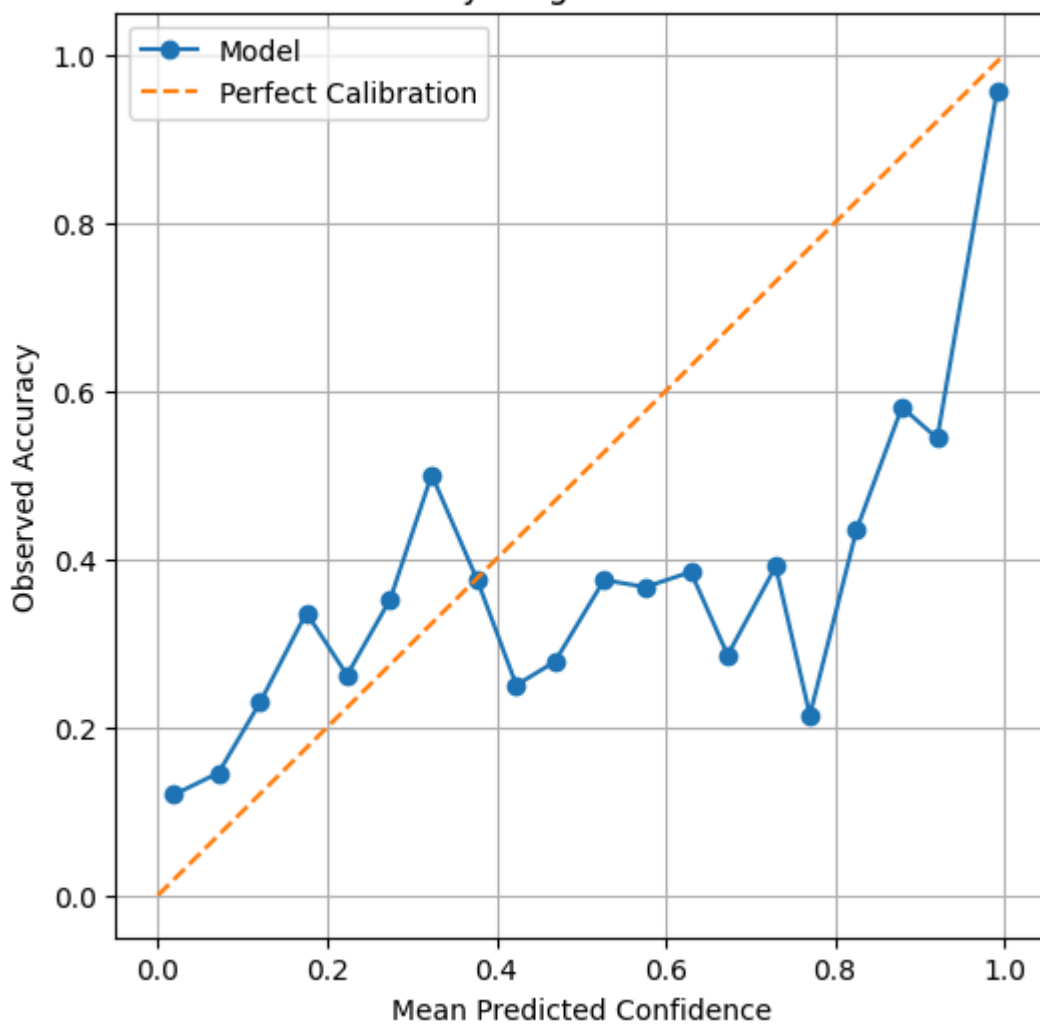
In [13]: # Cell 13
for model_name, (
    model,
    loader
) in models_dict.items():

    probs, preds, labels = collect_predictions(
        model,
        loader
    )

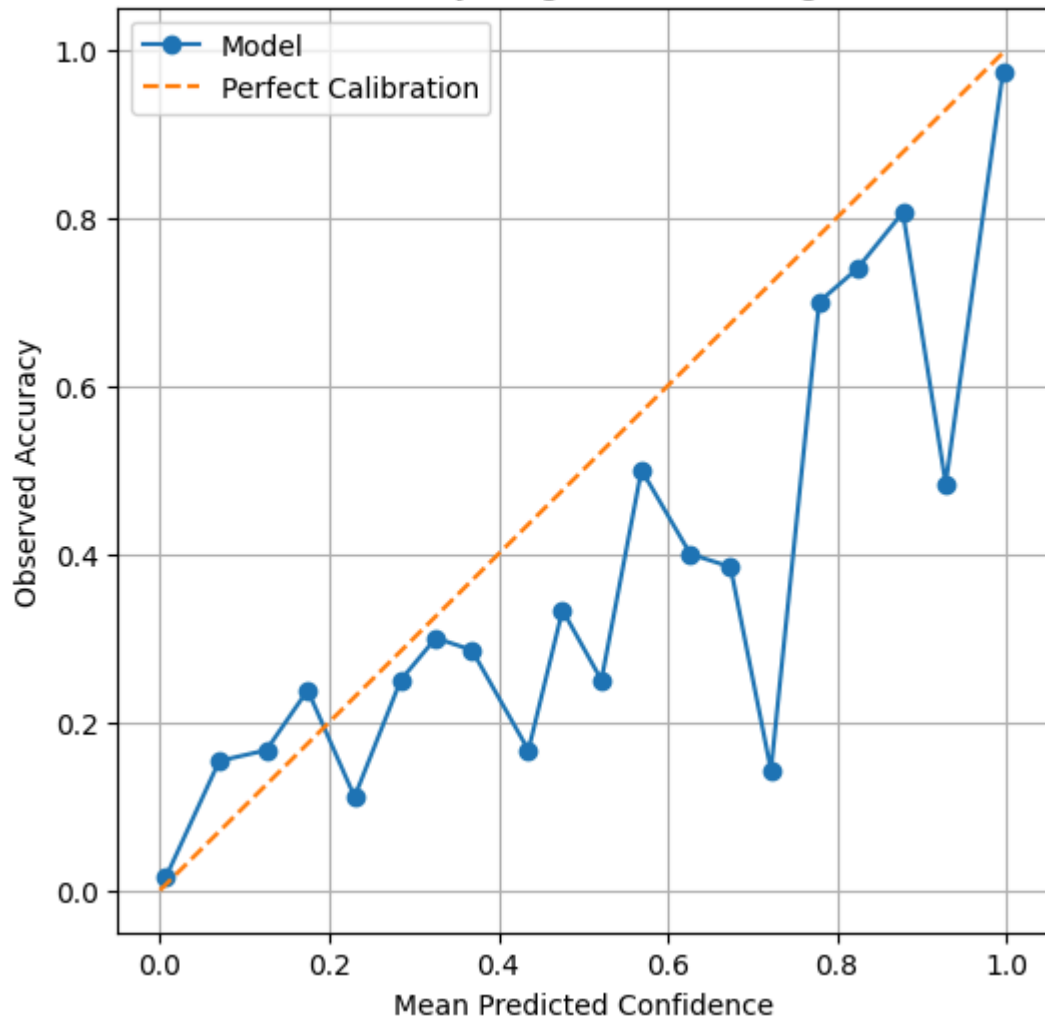
    plot_reliability_diagram(
        probs,
        labels,
        model_name
    )

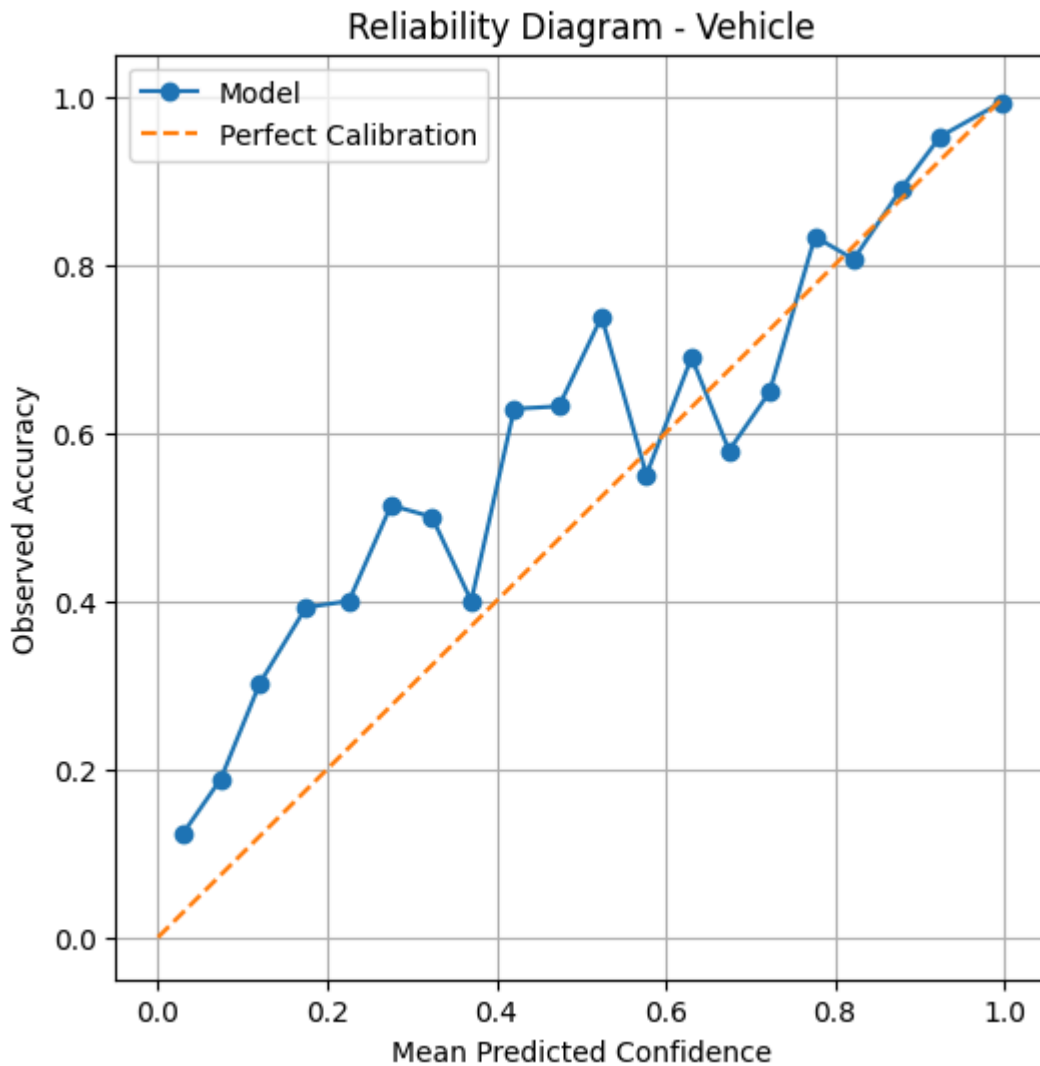
```

Reliability Diagram - Pedestrian



Reliability Diagram - Traffic Light





Exercise 7.5: Temperature Scaling

1. Apply temperature scaling to each of your three models. Optimize T on the validation

set using negative log-likelihood. 2. Report the ECE before and after scaling for each model.

```
In [14]: # Cell 14
validation_pedestrian = CarlaDataset(
    csv_file="../dataset/validation/labels.csv",
    img_dir="../dataset/validation/rgb-front",
    label_column="has_pedestrian",
    transform=transform
)

validation_traffic = CarlaDataset(
    csv_file="../dataset/validation/labels.csv",
    img_dir="../dataset/validation/rgb-front",
    label_column="has_traffic_light",
    transform=transform
)

validation_vehicle = CarlaDataset(
    csv_file="../dataset/validation/labels.csv",
```

```
img_dir="./dataset/validation/rgb-front",
label_column="has_vehicle",
transform=transform
)
```

```
In [15]: # Cell 15
validation_pedestrian_loader = DataLoader(
    validation_pedestrian,
    batch_size=32,
    shuffle=False
)

validation_traffic_loader = DataLoader(
    validation_traffic,
    batch_size=32,
    shuffle=False
)

validation_vehicle_loader = DataLoader(
    validation_vehicle,
    batch_size=32,
    shuffle=False
)
```

```
In [16]: # Cell 16
def find_best_temperature(
    model,
    loader
):

    model.eval()

    criterion = nn.BCEWithLogitsLoss()

    # -----
    # Compute logits ONLY ONCE
    # -----

    all_logits = []
    all_labels = []

    with torch.no_grad():

        for images, labels in loader:

            images = images.to(device)

            logits = model(images)

            all_logits.append(
                logits.cpu()
            )

            all_labels.append(
                labels.unsqueeze(1)
            )

    all_logits = torch.cat(all_logits)
```

```

all_labels = torch.cat(all_labels)

best_temperature = 1.0
best_loss = float("inf")

# -----
# Search Temperature
# -----

for T in np.arange(
    0.5,
    3.1,
    0.1
):

    scaled_logits = all_logits / T

    loss = criterion(
        scaled_logits,
        all_labels
    ).item()

    if loss < best_loss:

        best_loss = loss
        best_temperature = T

return (
    best_temperature,
    best_loss
)

```

```

In [17]: # Cell 17
validation_models = {

    "Pedestrian": (
        pedestrian_model,
        validation_pedestrian_loader
    ),

    "Traffic Light": (
        traffic_light_model,
        validation_traffic_loader
    ),

    "Vehicle": (
        vehicle_model,
        validation_vehicle_loader
    )
}

temperature_results = []

for model_name, (
    model,
    loader
) in validation_models.items():

    print(f"Finding T for {model_name}...")

```

```

    T, loss = find_best_temperature(
        model,
        loader
    )

    temperature_results.append([
        model_name,
        T,
        loss
    ])

temperature_df = pd.DataFrame(

    temperature_results,

    columns=[
        "Model",
        "Best Temperature",
        "Validation NLL"
    ]
)

temperature_df

```

Finding T for Pedestrian...
 Finding T for Traffic Light...
 Finding T for Vehicle...

Out[17]:

	Model	Best Temperature	Validation NLL
0	Pedestrian	2.9	0.525660
1	Traffic Light	1.1	0.065361
2	Vehicle	1.6	0.278473

In [18]:

```

# Cell 18
temperature_lookup = {
    row["Model"]: row["Best Temperature"]
    for _, row in temperature_df.iterrows()
}

temperature_lookup

```

Out[18]:

```

{'Pedestrian': 2.8999999999999995,
 'Traffic Light': 1.0999999999999999,
 'Vehicle': 1.5999999999999996}

```

In [19]:

```

# Cell 19
comparison = []
reliability_data = {}

for model_name, (
    model,
    loader
) in models_dict.items():

    # Before

    probs_before, preds_before, labels = collect_predictions(

```

```

        model,
        loader,
        temperature=1.0
    )

    ece_before = compute_ece(
        probs_before,
        preds_before,
        labels
    )

    # After

    probs_after, preds_after, labels = collect_predictions(
        model,
        loader,
        temperature_lookup[
            model_name
        ]
    )

    ece_after = compute_ece(
        probs_after,
        preds_after,
        labels
    )

    comparison.append([model_name, ece_before, ece_after, ece_before - ece_after])
    reliability_data[model_name] = (probs_before, probs_after, labels)

comparison_df = pd.DataFrame(

    comparison,

    columns=[

        "Model",

        "ECE Before",

        "ECE After",

        "Improvement"

    ]
)

comparison_df

```

Out[19]:

	Model	ECE Before	ECE After	Improvement
--	-------	------------	-----------	-------------

0	Pedestrian	0.107468	0.074552	0.032916
1	Traffic Light	0.036597	0.033935	0.002661
2	Vehicle	0.046837	0.021811	0.025026

In [20]:

```

# Cell 19.1
def plot_reliability_comparison(probs_before, probs_after, labels, model_name):

```

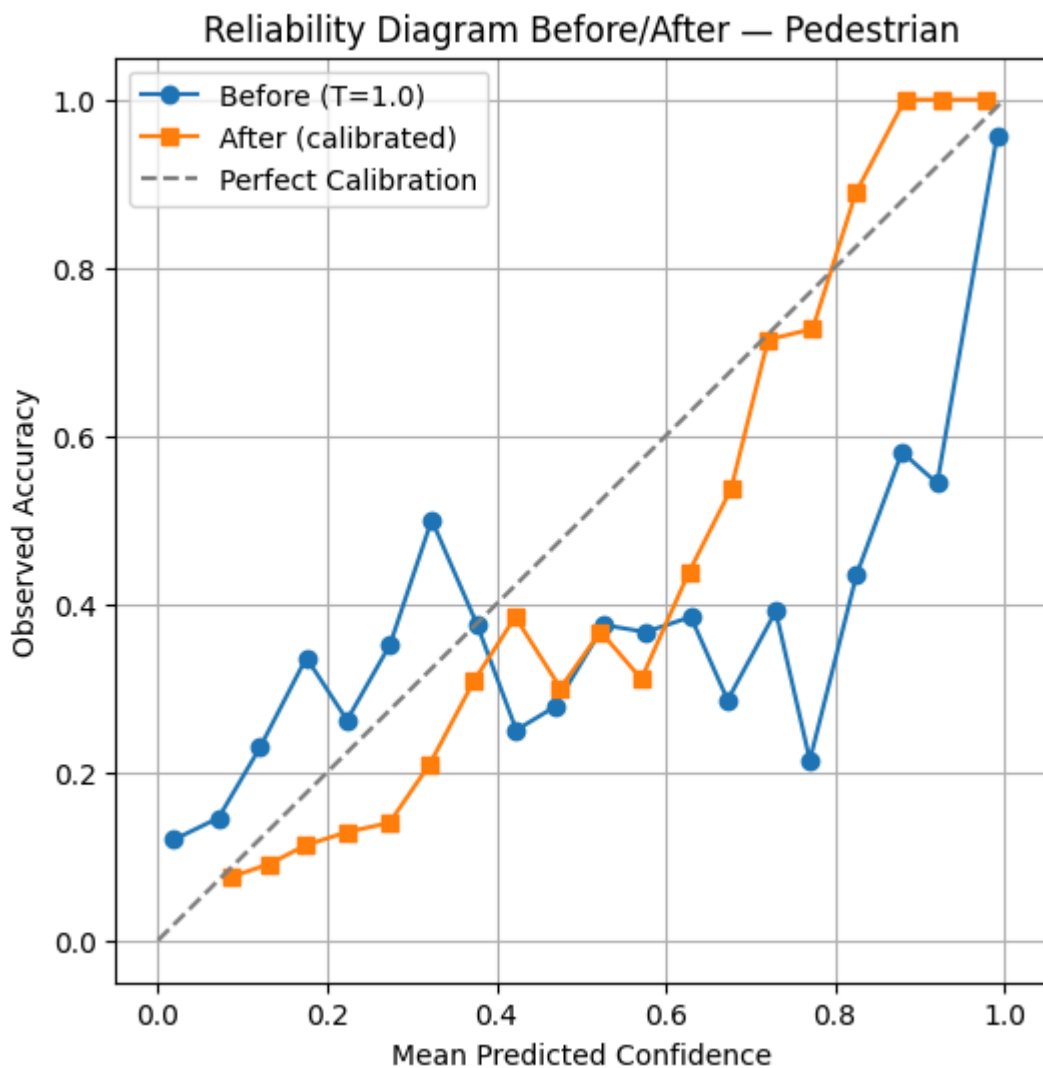
```

true_before, pred_before = calibration_curve(labels, probs_before, n_bins=20)
true_after, pred_after = calibration_curve(labels, probs_after, n_bins=20, s

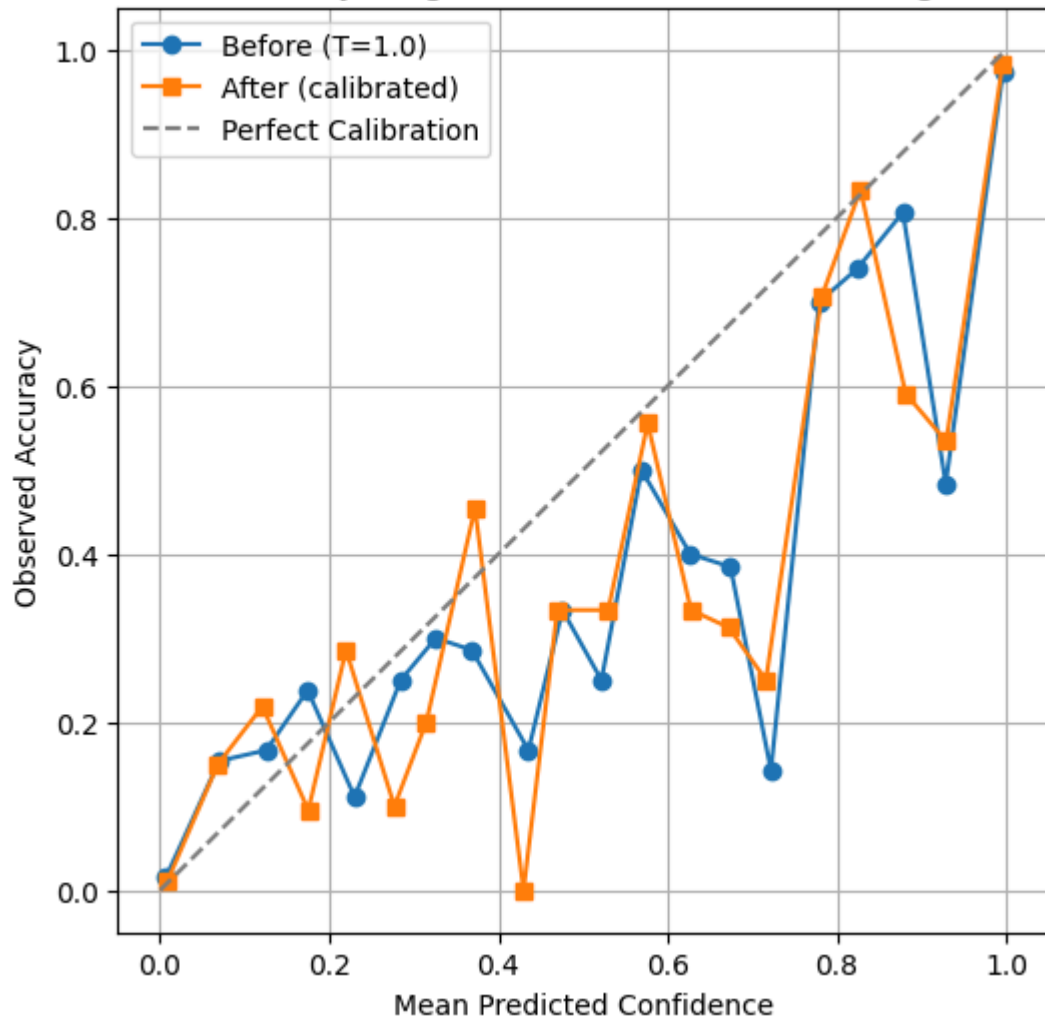
plt.figure(figsize=(6,6))
plt.plot(pred_before, true_before, marker="o", label="Before (T=1.0)")
plt.plot(pred_after, true_after, marker="s", label="After (calibrated)")
plt.plot([0,1], [0,1], "--", color="gray", label="Perfect Calibration")
plt.xlabel("Mean Predicted Confidence")
plt.ylabel("Observed Accuracy")
plt.title(f"Reliability Diagram Before/After - {model_name}")
plt.legend()
plt.grid(True)
plt.show()

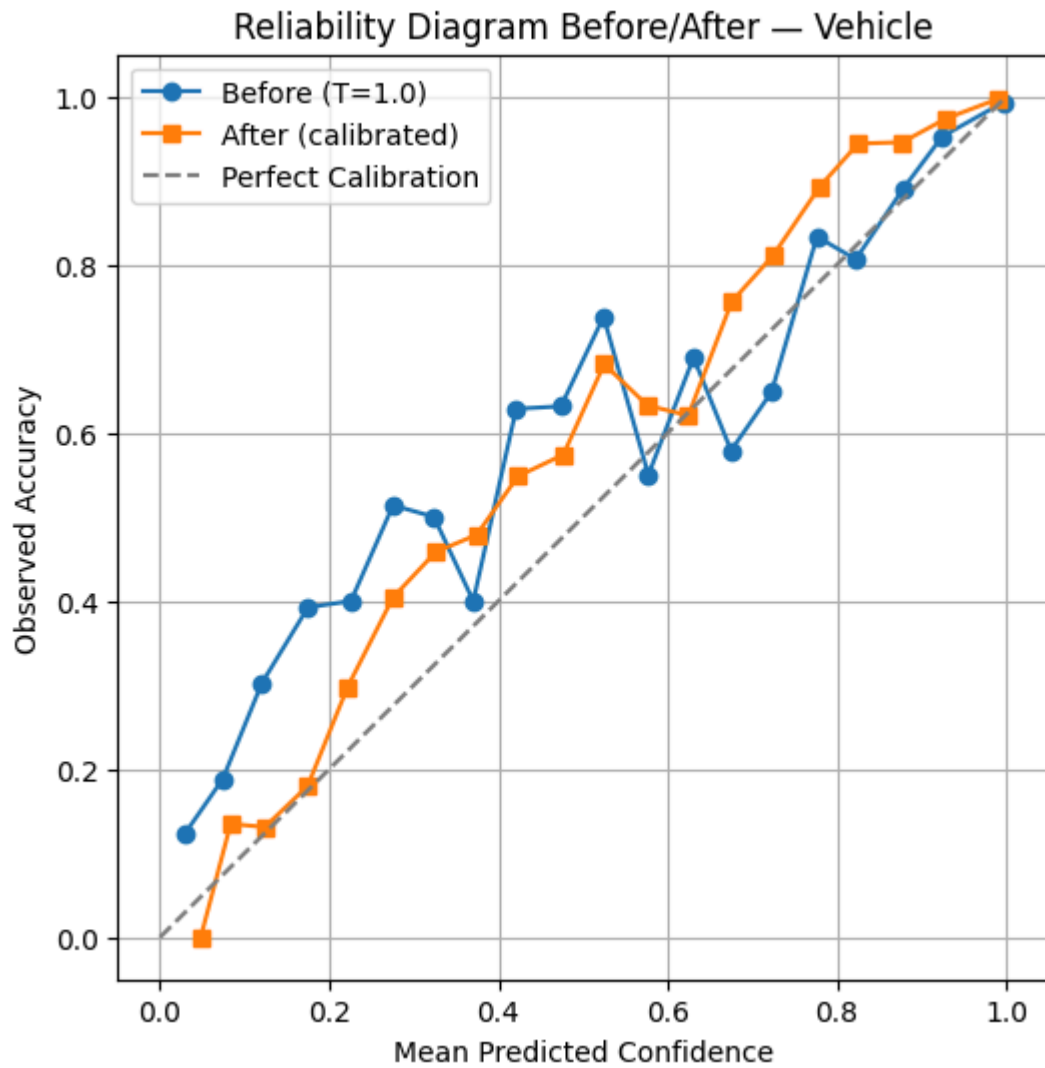
for model_name, (probs_before, probs_after, labels) in reliability_data.items():
    plot_reliability_comparison(probs_before, probs_after, labels, model_name)

```

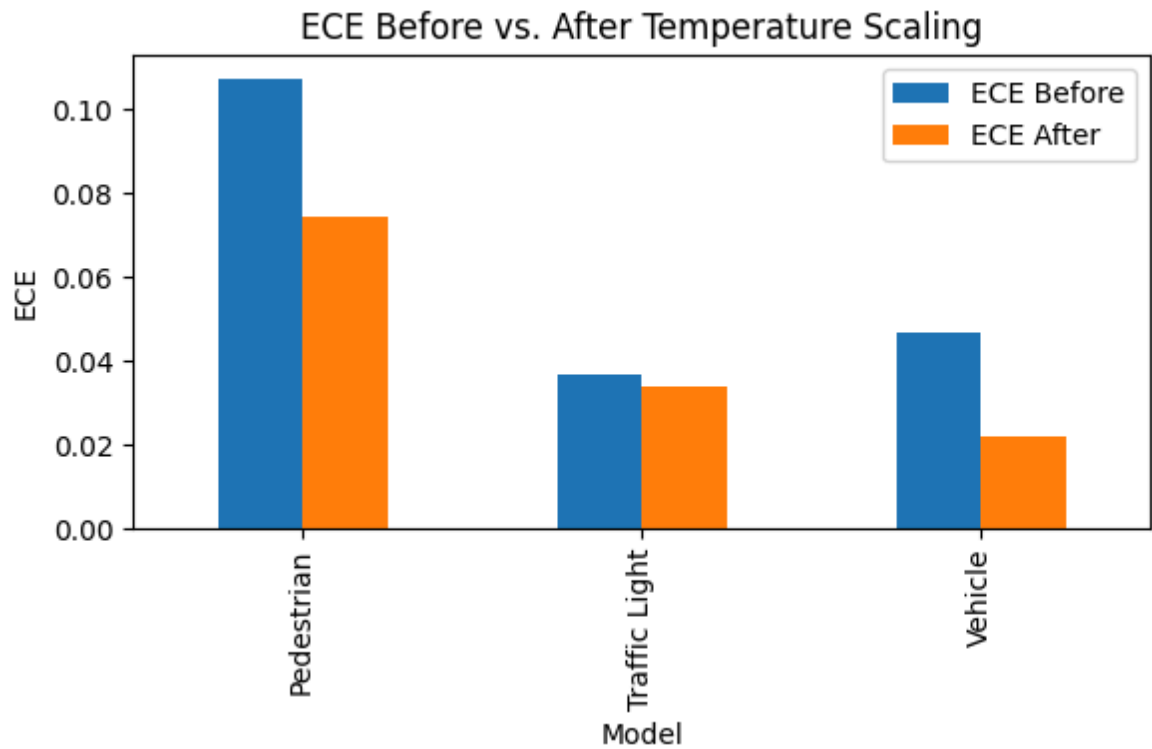


Reliability Diagram Before/After — Traffic Light





```
In [21]: # Cell 19.2
comparison_df.set_index("Model")[["ECE Before", "ECE After"]].plot(kind="bar", f
plt.ylabel("ECE")
plt.title("ECE Before vs. After Temperature Scaling")
plt.tight_layout()
plt.show()
```



```
In [22]: # Cell 20
def compute_total_loss(
    probs,
    labels,
    threshold,
    c_fn=100,
    c_fp=1
):
    preds = (
        probs >= threshold
    ).astype(int)

    false_negatives = np.sum(
        (preds == 0) &
        (labels == 1)
    )

    false_positives = np.sum(
        (preds == 1) &
        (labels == 0)
    )

    total_loss = (
        c_fn * false_negatives +
        c_fp * false_positives
    )

    return (
        total_loss,
        false_negatives,
        false_positives
    )
```

```
In [23]: # Cell 21
```

```

ped_probs_uncal, ped_preds, ped_labels = collect_predictions(
    pedestrian_model,
    pedestrian_loader,
    temperature=1.0
)

ped_temperature = temperature_lookup["Pedestrian"]

ped_probs_cal, _, _ = collect_predictions(
    pedestrian_model,
    pedestrian_loader,
    temperature=ped_temperature
)

```

```

In [24]: # Cell 22
threshold_default = 0.5
threshold_optimal = 0.0099

loss_uncal_05, fn1, fp1 = compute_total_loss(
    ped_probs_uncal,
    ped_labels,
    threshold_default
)

loss_uncal_opt, fn2, fp2 = compute_total_loss(
    ped_probs_uncal,
    ped_labels,
    threshold_optimal
)

loss_cal_05, fn3, fp3 = compute_total_loss(
    ped_probs_cal,
    ped_labels,
    threshold_default
)

loss_cal_opt, fn4, fp4 = compute_total_loss(
    ped_probs_cal,
    ped_labels,
    threshold_optimal
)

```

```

In [25]: # Cell 23
loss_table = pd.DataFrame(

    {

        "τ = 0.5": [
            loss_uncal_05,
            loss_cal_05
        ],

        "τ = 0.0099": [
            loss_uncal_opt,
            loss_cal_opt
        ]

    },

```

```

    index=[
        "Uncalibrated",
        "Calibrated"
    ]
)

loss_table

```

Out[25]:

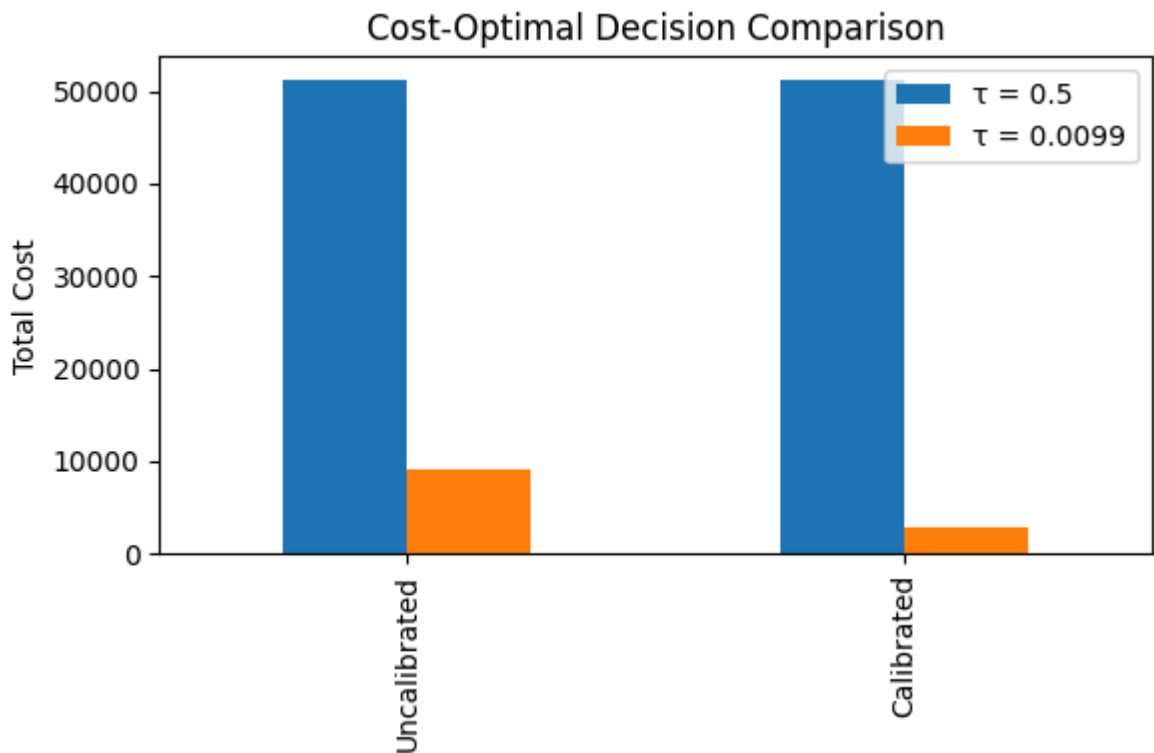
	$\tau = 0.5$	$\tau = 0.0099$
Uncalibrated	51156	9043
Calibrated	51156	2894

Uncalibrated	51156	9043
Calibrated	51156	2894

```

In [26]: # Cell 23.1
loss_table.plot(kind="bar", figsize=(6,4))
plt.ylabel("Total Cost")
plt.title("Cost-Optimal Decision Comparison")
plt.tight_layout()
plt.show()

```



```

In [27]: # Cell 24
print("Uncalibrated ( $\tau = 0.5$ )")
print(f"FN = {fn1}")
print(f"FP = {fp1}")
print(f"Loss = {loss_uncal_05}")

print()

print("Uncalibrated ( $\tau = 0.0099$ )")
print(f"FN = {fn2}")
print(f"FP = {fp2}")
print(f"Loss = {loss_uncal_opt}")

```

```
print()

print("Calibrated ( $\tau$  = 0.5)")
print(f"FN = {fn3}")
print(f"FP = {fp3}")
print(f"Loss = {loss_cal_05}")

print()

print("Calibrated ( $\tau$  = 0.0099)")
print(f"FN = {fn4}")
print(f"FP = {fp4}")
print(f"Loss = {loss_cal_opt}")
```

Uncalibrated (τ = 0.5)

FN = 510

FP = 156

Loss = 51156

Uncalibrated (τ = 0.0099)

FN = 68

FP = 2243

Loss = 9043

Calibrated (τ = 0.5)

FN = 510

FP = 156

Loss = 51156

Calibrated (τ = 0.0099)

FN = 0

FP = 2894

Loss = 2894